3. Web API. REST. Получение данных

Цель работы

Научиться отдавать данные из базы по протоколу HTTP. Созданное приложение развивается до конца семестра: четвёртая работа добавляет к нему изменение данных, пятая события, шестая интерфейс пользователя.

Материалы для подготовки

- Web API в ASP.NET Core: https://learn.microsoft.com/aspnet/core/web-api/
- Маршрутизация в контроллерах: https://learn.microsoft.com/aspnet/core/mvc/controllers/routing
- Методы HTTP: https://developer.mozilla.org/docs/Web/HTTP/Methods
- Коды состояния HTTP: https://developer.mozilla.org/docs/Web/HTTP/Status
- Swagger: https://learn.microsoft.com/aspnet/core/tutorials/web-api-help-pages-using-swagger
- Контроллеры и данные: https://metanit.com/sharp/mvc/12.1.php

Теоретические сведения

Что такое REST в объёме работы

Данные приложения рассматриваются как ресурсы. У каждого ресурса есть адрес, а действие над ним задаётся методом HTTP, а не самим адресом. Адрес вида `/api/books/delete/5` архитектуре REST не соответствует: действие оказалось в адресе.

Обращение к коллекции и обращение к одному элементу различаются адресом:

```
GET /api/books      список всех книг
GET /api/books/5    книга с идентификатором 5
```

Результат каждого обращения содержит код состояния. Код сообщает, что произошло, и обрабатывается клиентом независимо от тела ответа.

Примеры этого раздела построены на другой предметной области: книгах и издательствах. Переложить их на автомобили и дилеров предстоит самостоятельно.

Контроллер

Контроллер отвечает за один ресурс. Атрибут маршрута задаёт общий префикс адресов, а атрибуты методов уточняют остаток пути.

```
[ApiController]
[Route("api/[controller]")]
public class BooksController : ControllerBase
{
    [HttpGet]
    public ActionResult<IEnumerable<Book>> GetAll()
    {
        return Ok(repository.GetAll());
    }
```

```
    [HttpGet("{id}")]
    public ActionResult<Book> GetById(int id)
    {
        var book = repository.GetById(id);
        if (book is null)
        {
            return NotFound();
        }
        return Ok(book);
    }
}
```

Имя контроллера подставляется в маршрут вместо `[controller]` без окончания `Controller`, поэтому адреса начинаются с `/api/books`.

Коды ответа

Успешное чтение возвращает `200 OK` с телом ответа. Запрос несуществующего идентификатора возвращает `404 Not Found` с пустым телом.

Пустой список это не ошибка. Если в таблице нет ни одной записи, ответом будет `200 OK` с пустым массивом, а не `404`.

Возвращать `200 OK` с текстом ошибки внутри тела не следует: клиент проверяет код, а не содержимое.

Отделение доступа к данным

Запросы к базе не пишутся в теле метода контроллера. Работа с данными выносится в отдельный класс, а контроллер вызывает его методы.

```
public class BookRepository
{
    public IEnumerable<Book> GetAll() { /* ... */ }
    public Book GetById(int id) { /* ... */ }
}
```

Разделение понадобится в пятой работе: события добавляются именно к этому классу, и контроллер при этом не меняется.

Отладка

Проект Web API создаётся с поддержкой Swagger. Страница со списком методов открывается в обозревателе, запрос отправляется прямо с неё, там же видны адрес, код ответа и тело.

Postman решает ту же задачу и позволяет сохранять запросы, что удобно при повторной проверке.

> Идентификатор в адресе приходит из внешнего мира и может оказаться любым. Проверьте, как ведёт себя ваше приложение при запросе `/api/cars/0`, `/api/cars/-1` и `/api/cars/abc`.

Задания

1. Создайте проект Web API и убедитесь, что он запускается, а страница Swagger открывается.
2. Перенесите в проект классы предметной области из предыдущих работ.
3. Опишите классы доступа к данным для автомобилей и дилеров.
4. Добавьте контроллер автомобилей с методом, возвращающим все автомобили.
5. Добавьте в него метод, возвращающий один автомобиль по идентификатору.
6. Добавьте контроллер дилеров с теми же двумя методами.
7. Обеспечьте ответ `404` на запрос несуществующего идентификатора в обоих контроллерах.
8. Проверьте все четыре метода через Swagger или Postman и приведите результаты в отчёте.
9. Оформите отчёт по требованиям из общей части методических указаний.

Критерии приёмки

- Действие задаётся методом HTTP, а не словом в адресе.
- Адреса коллекции и отдельного элемента различаются идентификатором в пути.
- Запрос несуществующего идентификатора возвращает `404`, а не `200` с пустым телом.
- Пустая коллекция возвращается как `200` с пустым массивом.
- Запросы к базе находятся в отдельном классе, а не в теле метода контроллера.
- Студент показывает работу всех четырёх методов и называет коды ответа.

Контрольные вопросы

1. Чем ресурс отличается от действия и почему адрес `/api/cars/delete/5` не соответствует REST?
2. Как по адресу запроса отличить обращение к коллекции от обращения к элементу?
3. Что означают коды `200`, `404` и `400`?
4. Почему пустой список это не `404`?
5. Что подставляется в маршрут вместо `[controller]`?
6. Зачем работа с данными вынесена из контроллера в отдельный класс?
7. Что вернёт ваше приложение на запрос с нечисловым идентификатором и на каком этапе это происходит?